

GUÍA DE APRENDIZAJE: Arquitectura Backend

Persistencia de Datos, Capas y Seguridad Zero-Trust

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Programa de Formación:** Arquitectura Backend: Persistencia de Datos, Capas y Seguridad Zero-Trust.
- **Competencia:** Construcción de software, gestión de bases de datos y seguridad perimetral.
- **Resultados de Aprendizaje:**
 1. Aislar credenciales e infraestructura mediante Variables de Entorno (`.env`) bajo la metodología 12-Factor App.
 2. Comprender el ciclo de vida del orquestador (`app`) y su conexión con las capas del sistema.
 3. Aplicar los 4 pilares de la Programación Orientada a Objetos (POO) en el modelado de bases de datos.
 4. Implementar un ORM (SQLAlchemy) gestionando la evolución del esquema (Migraciones).
 5. Ejecutar operaciones transaccionales (CRUD) aplicando Hashing criptográfico y Paginación.
 6. Asegurar endpoints críticos mediante el estándar JWT y Control de Acceso Basado en Roles (RBAC).
- **Duración:** 80 horas.

2. PRESENTACIÓN: La Arquitectura Real y el Modelo Zero-Trust

Hasta el Nivel 3, los programas desarrollados eran pequeños scripts que vivían en la memoria RAM (volatilidad total). Si se iba la luz, el inventario desaparecía. Además, la configuración estaba "quemada" en el código (hardcoded) y cualquier persona con Postman podía alterar el sistema. Un Arquitecto de Software no permite esto.

En esta guía daremos el salto hacia la **Arquitectura de Capas y la Persistencia Segura**.

Aprenderás cómo diferentes archivos se conectan entre sí para formar un todo cohesionado:

1. Blindaremos la configuración basándonos en la metodología 12-Factor App (Variables de entorno).

2. Aplicaremos la POO usando **SQLAlchemy (ORM)** para traducir Clases a Tablas en el disco duro, controlando su evolución con **Flask-Migrate**.
3. Protegeremos nuestro dominio integrando **Hashing Criptográfico** y **JWT (JSON Web Tokens)** para operar bajo un modelo **Zero-Trust** (Confianza Cero).

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

FASE 1: El Orquestador Central y el Aislamiento de Infraestructura (`app.py` y `.env`)

Concepto 1: Aislamiento (`.env`) El principio número 3 de la metodología 12-Factor App establece que la configuración estricta debe guardarse en el entorno, **nunca en el código**. El código se comparte en Git; las contraseñas no. Usamos la librería `python-dotenv` para leer un archivo oculto llamado `.env` que inyecta las variables en la memoria del Sistema Operativo.

Concepto 2: ¿Qué es realmente `app` ? Para un ingeniero, `app` no es magia; es un **Objeto Instanciado** que actúa como el "Director de Orquesta" (Patrón Singleton). Guarda configuraciones secretas, sabe qué URL conecta con qué función, y ensambla la Base de Datos con el servidor web.

Archivos de Configuración

Crea un archivo llamado `.env` en la raíz de tu proyecto. (Nota: Este archivo **NO** lleva comillas en los valores).

```
# Archivo: .env
FLASK_DEBUG=True
PORT=5000

# Infraestructura de Datos (URI de conexión a la BD)
DATABASE_URI=sqlite:///trapiche_db.sqlite

# Seguridad y Criptografia (Llave maestra del servidor)
JWT_SECRET_KEY=firma_super_secretaria_desarrollo_123_!@#
```

Crea el archivo `.gitignore` para instruir a Git que ignore archivos sensibles y de caché:

```
# Archivo: .gitignore
.env
__pycache__/
*.sqlite
```

El Orquestador Central (`app.py`)

```

# Archivo: app.py
import os
from flask import Flask
from dotenv import load_dotenv

# 1. Cargamos el archivo .env a la memoria del Sistema Operativo
load_dotenv()

# 2. [INSTANCIACIÓN]: Creamos el Orquestador Central
app = Flask(__name__)

# 3. [ENCAPSULAMIENTO DE CONFIGURACIÓN]: Guardamos los secretos dentro de 'app'
# Si la variable no existe en el .env, usamos un valor fallback por seguridad.
app.config['SQLALCHEMY_DATABASE_URI'] = os.getenv('DATABASE_URI', 'sqlite:///defc
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['JWT_SECRET_KEY'] = os.getenv('JWT_SECRET_KEY', 'clave-insegura')

# (En las siguientes fases conectaremos la Base de Datos y la Seguridad aquí)

if __name__ == '__main__':
    # El archivo app.py es el único que arranca el servidor
    puerto = int(os.getenv('PORT', 5000))
    modo_debug = os.getenv('FLASK_DEBUG') == 'True'
    app.run(port=puerto, debug=modo_debug)

```

FASE 2: La Capa de Datos, Migraciones y Pilares POO (`models.py`)

Concepto: Las bases de datos hablan SQL; tu código habla Python. Un ORM (Object-Relational Mapping) traduce automáticamente tus objetos a registros físicos en disco. Separamos esto en otro archivo por el principio de Responsabilidad Única.

Explicación Detallada (Pilares POO y Migraciones):

- **Abstracción:** Ocultar la complejidad de SQL.
- **Herencia:** Nuestras tablas heredan de `db.Model`, obteniendo métodos de guardado y búsqueda preprogramados.
- **Encapsulamiento:** Ocultamos contraseñas reales detrás de un Hash.
- **Sobre `db.create_all()`:** Usar esto en producción es un error crítico porque no actualiza tablas existentes. Usaremos **Migraciones (Alembic)**, un control de versiones para la base de datos que altera tablas sin destruir datos.

Capa de Lógica de Datos (`models.py`)

```

# Archivo: models.py
from flask_sqlalchemy import SQLAlchemy

```

```
# Instanciamos la herramienta vacía. Se acoplará a 'app' más adelante.
db = SQLAlchemy()

# [HERENCIA]: La clase 'Usuario' hereda todo el poder de 'db.Model'
class Usuario(db.Model):
    __tablename__ = 'usuarios'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)

    # [ENCAPSULAMIENTO]: Nunca guardamos el password real, solo su representación
    password_hash = db.Column(db.String(255), nullable=False)
    rol = db.Column(db.String(20), default='Operario')

    # [POLIMORFISMO]: Método propio para transformar el objeto a JSON
    def serializar(self) -> dict:
        return {
            "id": self.id,
            "username": self.username,
            "rol": self.rol
        }
```

Conectando `models.py` con `app.py`

Regresamos a `app.py` para inyectar el Orquestador en el ORM. *Añade esto antes del `if`*

```
__name__ == '__main__':
```

```
# Agregar en app.py (Sección de Acoplamiento)
from models import db
from flask_migrate import Migrate
from flask_jwt_extended import JWTManager

# [ACOPLAMIENTO]: Le pasamos el orquestador 'app' al ORM y utilidades
db.init_app(app)
migrate = Migrate(app, db) # Motor de migraciones
jwt = JWTManager(app)      # Motor de seguridad JWT
```

(En la terminal ejecutarás: `flask db init`, luego `flask db migrate -m "Inicio"`, y `flask db upgrade` para escribir el esquema en disco).

FASE 3: Capa de Enrutamiento, Hashing y Transacciones (`routes.py`)

Concepto: Ya tenemos la BD y el Servidor. Faltan las "Puertas de Entrada" (Endpoints).

Crearemos `routes.py` como la **Capa de Controladores**.

Explicación Detallada (Transacciones y Seguridad):

1. Los **Blueprints** le dicen al Orquestador qué hacer cuando llega una petición a una URL específica.
2. Las BD operan con **Transacciones ACID**. Los cambios ocurren en RAM (`db.session.add()`). Solo al hacer `db.session.commit()` se escriben en disco. Si falla, `db.session.rollback()` evita la corrupción de datos.
3. Usamos `generate_password_hash` para destruir la contraseña real y guardar un código matemático ininteligible en la BD.

CRUD Seguro (`routes.py`)

```
# Archivo: routes.py
from flask import Blueprint, request, jsonify, Response
from werkzeug.security import generate_password_hash, check_password_hash
from flask_jwt_extended import create_access_token, jwt_required, get_jwt_identity
from models import db, Usuario

# Blueprint: Un mini-orquestador para agrupar rutas y exportarlas a app.py
api_bp = Blueprint('api', __name__)

@api_bp.route('/usuarios/regar', methods=['POST'])
def registrar_usuario() -> tuple[Response, int]:
    try:
        payload = request.get_json()

        # 1. Hashing Criptográfico en RAM
        clave_segura = generate_password_hash(payload['password'])

        # 2. [INSTANCIACIÓN]: Objeto en RAM
        nuevo_user = Usuario(
            username=payload['username'],
            password_hash=clave_segura,
            rol=payload.get('rol', 'Operario')
        )

        # 3. [ABSTRACCIÓN TRANSACCIONAL]: Guardamos en disco
        db.session.add(nuevo_user)
        db.session.commit() # Consolidación atómica

        return jsonify({"mensaje": "Éxito", "data": nuevo_user.serialize()}), 200

    except Exception as e:
        db.session.rollback() # Revierte la RAM si hubo un fallo (ej. username du
        return jsonify({"error": "Fallo de integridad", "detalle": str(e)}), 400

@api_bp.route('/usuarios', methods=['GET'])
def listar_usuarios() -> tuple[Response, int]:
    # Paginación: Protege la RAM para no cargar 1 millón de registros de golpe
    pagina = request.args.get('page', 1, type=int)
    paginacion = Usuario.query.paginate(page=pagina, per_page=10, error_out=False)
```

```

resultado = [u.serializar() for u in paginacion.items]

return jsonify({
    "pagina_actual": paginacion.page,
    "total_paginas": paginacion.pages,
    "usuarios": resultado
}), 200

```

Recuerda registrar el Blueprint en tu `app.py`: `app.register_blueprint(api_bp, url_prefix='/api')`

FASE 4: El Guardián del Castillo (Fundamentos de JWT)

Concepto: Un JWT (JSON Web Token) es un "carnet digital" criptográficamente firmado que el servidor le entrega al cliente tras validar su contraseña.

Explicación Detallada (El Por Qué Técnico): Las APIs REST son "Stateless" (Sin Estado). El servidor no gasta RAM recordando quién está logueado. El JWT contiene la información del usuario (Payload) firmada con la `JWT_SECRET_KEY`. Si el token es alterado, la firma matemática se rompe y el servidor lo rechaza en tiempo $O(1)$.

Agregando Lógica de Login en `routes.py`

```

# Agregar en routes.py
@api_bp.route('/login', methods=['POST'])
def login() -> tuple[Response, int]:
    payload = request.get_json()

    # 1. Buscamos al usuario en el disco
    usuario = Usuario.query.filter_by(username=payload.get('username')).first()

    # 2. Validación de hashes (Compara texto plano vs hash almacenado)
    if usuario and check_password_hash(usuario.password_hash, payload.get('password')):
        # 3. Generamos Token inyectando la identidad (Rol)
        identidad = {"username": usuario.username, "rol": usuario.rol}
        token_acceso = create_access_token(identity=identidad)

        return jsonify({"mensaje": "Login exitoso", "token": token_acceso}), 200

    return jsonify({"error": "Credenciales inválidas"}), 401

```

FASE 5: Protegiendo el Perímetro (Decoradores y RBAC)

Concepto: Autenticar es saber *quién eres*. Autorizar es saber *qué puedes hacer* (Rol).

Protegemos las rutas usando el patrón Decorador (`@`).

Explicación Detallada: Al poner `@jwt_required()` encima de una función, la petición HTTP es interceptada. Flask busca la cabecera `Authorization: Bearer <token>`. Si no existe o la firma es falsa, aborta con error 401.

Agregando Endpoint Crítico en `routes.py`

```
# Agregar en routes.py
@api_bp.route('/inventario/critico', methods=['POST'])
@jwt_required() # BARRERA 1: Bloquea peticiones no autenticadas (Sin Token)
def modificar_inventario() -> tuple[Response, int]:
    # Extraemos el diccionario de identidad inyectado en el Token
    usuario_actual = get_jwt_identity()

    # BARRERA 2: Control de Acceso Basado en Roles (RBAC)
    if usuario_actual.get("rol") != "Admin":
        return jsonify({"error": "Forbidden: Requiere privilegios de Administrador"}), 403

    return jsonify({
        "mensaje": "Acceso concedido al servidor.",
        "operador": usuario_actual["username"]
    }), 200
```

FASE 6: Integración Empresarial (Reto Integrador)

[EVIDENCIA DE PRODUCTO FINAL] - Arquitectura Modular Backend:

1. Crea la estructura modular: `.env`, `.gitignore`, `models.py`, `routes.py` y `app.py`.
2. En `models.py`: Crea las clases `Usuario` y `Producto` mapeadas a la BD.
3. En `app.py`: Ensambla el proyecto (`init_app`), inicializa JWT y registra el Blueprint.
4. Ejecuta las migraciones en terminal (`flask db init`, `migrate`, `upgrade`) para crear la base de datos física.
5. En `routes.py`: Implementa el registro con Hashing, el Login con JWT, y una ruta de creación de Productos protegida con `@jwt_required()` que verifique el rol "Admin".
6. Documenta en Postman los flujos:
 - Error 401 (Petición sin token).
 - Login exitoso (Status 200).
 - Error 403 (Intento con token de rol 'Operario' a una ruta Admin).
 - Inserción exitosa en BD (Status 201). Exporta la colección de Postman.

HERRAMIENTAS DE DEPURACIÓN: ¿Cómo ver los datos en SQLite?

Al utilizar un ORM, es vital verificar que los datos se están guardando correctamente en el archivo físico. En tu entorno de desarrollo IDE (**Antigravity** o VS Code standard), tienes dos formas principales de inspeccionar el archivo `.sqlite`:

Opción 1: Interfaz Visual (Recomendada)

1. Ve a la pestaña de Extensiones en tu entorno Antigravity.
2. Busca e instala "**SQLite Viewer**" (de Florian Klampfer) o una extensión equivalente permitida en tu entorno.
3. Haz clic sobre tu archivo `trapiche_db.sqlite` en el explorador de archivos. Se abrirá una pestaña visual donde podrás explorar las tablas y filas como si fuera un Excel.

Opción 2: Terminal Integrada (Nativa y Universal) Si las políticas de red/extensions de Antigravity restringen plugins visuales, siempre puedes usar la terminal nativa que viene preinstalada en entornos Linux/Desarrollo:

1. Abre la terminal (`Ctrl + ```).
2. Conéctate a tu base de datos escribiendo: `sqlite3 trapiche_db.sqlite`
3. Usa el comando `.tables` para ver qué tablas creó el ORM.
4. Usa SQL estándar para ver la data: `SELECT * FROM usuarios;`
5. Escribe `.quit` para salir.

4. PREPARACIÓN PARA LA EVIDENCIA DE CONOCIMIENTO

Para el cuestionario técnico, el aprendiz debe dominar:

1. **La Instancia Orquestadora** (`app`): Explicar el rol del objeto `app` y por qué herramientas como SQLAlchemy o JWT necesitan conectarse a él.
2. **Entornos y 12-Factor**: Justificar por qué subir un archivo `.env` a un repositorio público compromete toda la arquitectura del sistema.
3. **Pilares de la POO en el ORM**: Explicar cómo el ORM aplica la *Herencia* y la *Abstracción* para evitar escribir código SQL manual.
4. **Ciclo Transaccional ACID**: Entender por qué es obligatorio el uso de `commit()` (escribir en disco) y `rollback()` (revertir cambios en RAM en caso de excepción).
5. **Autenticación vs Autorización**: Diferenciar la emisión de un JWT (Autenticación) de la verificación del perfil del usuario mediante RBAC (Autorización).

5. GLOSARIO TÉCNICO

- **.env (Dotenv)**: Archivo que almacena configuración sensible para ser inyectada al Sistema Operativo de forma segura.

- **Singleton (Orquestador):** Patrón de diseño donde solo existe una única instancia de un objeto (la variable `app`).
- **ORM (Object-Relational Mapping):** Técnica para convertir datos entre el paradigma Orientado a Objetos (Python) y el relacional (Tablas SQL).
- **Transacción (Commit/Rollback):** Unidad lógica de procesamiento. O se ejecuta íntegra (`commit`) o se revierte a su estado seguro inicial (`rollback`).
- **JWT (JSON Web Token):** Estándar de transmisión Stateless. Contiene Header, Payload y Signature criptográfica.

6. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE

Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnica e Instrumento
AP10 - Arquitectura de Capas, Entornos y ORM. Desacoplar la aplicación en módulos lógicos aplicando pilares POO y gestión del <code>.env</code> .	1. Evidencia de Conocimiento: Cuestionario técnico sobre conexión de archivos (<code>app</code> , <code>models</code>), aplicación de POO en ORM, transacciones ACID y ciclo de seguridad JWT.	• Identifica el rol del Orquestador (<code>app</code>) y variables de entorno. • Reconoce la Herencia y Abstracción en la capa de datos. • Diferencia RAM de persistencia transaccional.	Técnica: Formulación de preguntas Instrumento: Cuestionario (LMS Zajuna)
AP11 - Transaccionalidad y Enrutamiento Modular. Desarrollar operaciones CRUD conectando controladores de red (<code>routes</code>) con modelos de disco (<code>models</code>).	2. Evidencia de Desempeño: Code Review ensamblando el orquestador, conectando variables de entorno, ejecutando migraciones y transacciones atómicas.	• Ensambla múltiples archivos <code>.py</code> mediante importaciones. • Mapea clases hacia tablas aplicando Encapsulamiento (Hash). • Gestiona el estado de transacciones (<code>commit</code> / <code>rollback</code>).	Técnica: Observación Sistemática (Code Review) Instrumento: Rúbrica de Desempeño Técnico
AP12 - Seguridad, Metaprogramación y RBAC. Implementar arquitecturas Zero-Trust	3. Evidencia de Producto: Entrega de archivo <code>.zip</code> conteniendo proyecto estructurado y	• Desarrolla una API REST modular, resiliente y escalable.	Técnica: Valoración de Producto

mediante decoradores de intercepción JWT.	exportación de pruebas Postman.	• Demuestra la aplicación del estándar Stateless JWT y políticas RBAC.	Instrumento: Lista de Verificación Arquitectónica y Postman
		Valida flujo de	